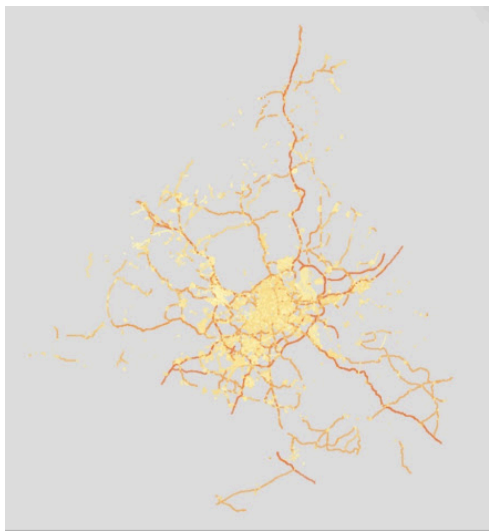


## Inferencia de velocidades máximas en red viaria de la Comunidad de Madrid

**Objetivo:** construir conjunto de datos que esten lo mas completos posibles con las velocidades maximas en tramos de la comunidad. Usable en aplicaciones de navegacion (waze, google maps, etc.), tambien para analizar la seguridad vial y para modelos de trafico y para poder detectar aumentos de velocidad sobre el maximo permitidop.

**El problema principal:** es que el mapa de etiqueta max no es completo.  
Etiqueta maxspeed no esta presente en todas las vias de la OSM



### **Análisis del mapa:**

- En el centro la cobertura de los tramos es alta, pero en municipios del area metropolitana por ejemplo San Sebastian de los Reyes les faltan bastantes etiquetas y luego en zonas mas rurales con secondary (carreteras secundarias) faltan aun mas etiquetas. Falta mucha informacion y muchos datos en ese tipo de zonas.

### **Soluciones:**

1. Si a un tramo le falta la etiqueta pero los tramos anterior y posterior de la misma carretera sí la tienen, se le podría asignar esa misma velocidad automáticamente para que luego el mapa pudiese verse un poco mas logico, el unico problema podriann ser las travesias que pueden estar en medio de dos tramos para cambiar de tramo rapido a tramo de población
- Efectivo por que en carreteras interurbanas, la velocidad es igual siempre y los tramos que no tengan etiquetas seran casos muy aislados. Tramos que hayan tenido bifurcación o hayan sido divididos por razones tecnicas de edicion en OSM, y en entradas o salidas de zonas urbanas ya que la velocidad ira incrememtandose poco a poco.

**Usando nodos:** localizar los tramos que no tengan maxspeed y ver los tramos vecinos a ella que compartan un nodo de inicio o un nodo de fin. Si se ve que todos los tramos vecinos tienen adjudicados un maxspeed del mismo valor, entonces asignaremos ese valor al tramo e iterar este proceso constantemente hasta que se vayan resolviendo diferentes maxspeed para diferentes tramos.

Basado en la OSM los tipos de tramos son: y DGT velocidad inferida:

(<https://download.geofabrik.de/osm-data-in-gis-formats-free.pdf>)

- motorway → autopista → 120km/h
- trunk → autovia → 120km/h
- primary → carretera principal → 90km/h
- secondary → carretera secundaria → 90km/h
- tertiary → carretera terciaria → 70km/h
- unclassified → no clasificada → 50km/h
- residential → residencial → 30km/h
- living\_street → peatonal → 20km/h
- service → via de servicio → 20km/h
- track → camino → 20km/h

| code | fclass         | Description                                                                                | OSM Tags               |
|------|----------------|--------------------------------------------------------------------------------------------|------------------------|
| 511x |                | <b>Major roads</b>                                                                         |                        |
| 5111 | motorway       | Motorway/freeway                                                                           | highway=motorway       |
| 5112 | trunk          | Important roads, typically divided                                                         | highway=trunk          |
| 5113 | primary        | Primary roads, typically national.                                                         | highway=primary        |
| 5114 | secondary      | Secondary roads, typically regional.                                                       | highway=secondary      |
| 5115 | tertiary       | Tertiary roads, typically local.                                                           | highway=tertiary       |
| 512x |                | <b>Minor Roads</b>                                                                         |                        |
| 5121 | unclassified   | Smaller local roads                                                                        | highway=unclassified   |
| 5122 | residential    | Roads in residential areas                                                                 | highway=residential    |
| 5123 | living_street  | Streets where pedestrians have priority                                                    | highway=living_street  |
| 5124 | pedestrian     | Pedestrian only streets                                                                    | highway=pedestrian     |
| 5125 | busway         | Dedicated roads for bus, usually closed for any mode of transport except public transport. | highway=busway         |
| 513x |                | <b>Highway links (sliproads/ramps)</b>                                                     |                        |
| 5131 | motorway_link  | Roads that connect from one road to another of the same or lower category.                 | highway=motorway_link  |
| 5132 | trunk_link     |                                                                                            | highway=trunk_link     |
| 5133 | primary_link   |                                                                                            | highway=primary_link   |
| 5134 | secondary_link |                                                                                            | highway=secondary_link |
| 5135 | tertiary_link  |                                                                                            | highway=tertiary_link  |

**2.** El objetivo de este proyecto es tener la velocidad maxima de cada tramo pero la OSM clasifica las vias en plan tertiary, seocndary, residential, etc. pero no siempre nos dice la velocidad real. Una via tipo tertiary (es una carretera local entre poblaciones) puede ser una calle de Mirasierra que va a 30km/h o tambien puede ser una via interurbana que va a 90km/h.

- El contexto geografico en el que esta el tramo es el problema ya que la misma carretera podria ser 30 en ciudad y 90 a las afueras, entonces si está en zona urbana recibira una velocidad urbana y lo mismo si esta fuera, recibira una velocidad interurbana

**Landuse:** es un poligono que tiene la etiqueta 'landuse' que basicamente lo que hace es cubrir zonas enteras. Los poligonos se han mapeado por la OSM y tienen buena cobertura donde hay mas escasez de datos. Pero el tramo esta en ciudad o en zona rural? Tendremos que cruzar dos capas geograficas en un **spatial join** (GIS) - Geographic information system

**Capa 1: Carreteras sin velocidad**

**Capa 2: Zonas de landuse que son los poligonos**

El proceso seria ver si hay algun tramo que no tenga la etiqueta de maxspeed y ver si intersecta con alguna zona de landuse del poligono urbano

- Si intersecta, se pondra la velocidad urbana segun el tipo de via
- SI no entersecta, se pondra una velocidad de velocidad interurbana segun el tipo

1. Los polígonos de **landuse** tienen buena cobertura precisamente en las zonas urbanas, que es exactamente donde necesitas distinguir si una tertiary va a 30 o a 90. Es una ventaja, no una coincidencia.

2. Los valores urbanos serían **residential, commercial, industrial, retail**. Los rurales serían **farmland, forest, meadow**. Si no intersecta con ningún polígono, se trata como rural por defecto.
3. Ccruzamos dos capas geográficas mediante un spatial join

El problema no es el tipo de vía, sino el contexto geográfico. Una **tertiary** (es una carretera local entre poblaciones) puede ser 30 km/h (travesía) en Mirasierra o 90 km/h (cc)

**La solución es cruzar las carreteras sin velocidad con los polígonos de landuse de OSM mediante un spatial join:**

**Capa 1: Carreteras sin velocidad**

**Capa 2: Zonas de landuse que son los poligonos**

si el tramo cae dentro de una zona urbana (residential, commercial, industrial) se le asigna velocidad urbana según su tipo. Si no intersecta con ninguna zona urbana, se le asigna velocidad interurbana. En caso de duda, interurbana por defecto.

```
gdf_vias = gpd.sjoin(gdf_vias, gdf_landuse, how="left", predicate="intersects")
gdf_vias["maxspeed_inferida"] = gdf_vias.apply(
    lambda x: velocidad_urbana(x.highway) if x.landuse in zonas_urbanas
    else velocidad_interurbana(x.highway), axis=1
)
```

| Zonas urbanas                                                                                                                      | Zonas rurales                                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>- landuse=comercial</li> <li>- landuse=industrial</li> <li>- landuse=residential</li> </ul> | <ul style="list-style-type: none"> <li>- landuse=forest</li> <li>- landuse=farmland</li> <li>- landuse=meadow</li> </ul> |

<https://wiki.openstreetmap.org/wiki/Key:landuse>

<https://wiki.openstreetmap.org/wiki/Key:maxspeed>

OSM usa ‘osm-legal-default-speeds’ que es una libreria que rellan

| OSM Tags                                                  |
|-----------------------------------------------------------|
| landuse=forest,<br>natural=wood                           |
| leisure=park,<br>leisure=common                           |
| landuse=residential                                       |
| landuse=industrial                                        |
| landuse=cemetery                                          |
| landuse=allotments                                        |
| landuse=meadow                                            |
| landuse=commercial                                        |
| leisure=recreation_ground or<br>landuse_recreation_ground |
| landuse=retail                                            |
| landuse=military                                          |
| landuse=quarry                                            |
| landuse=orchard                                           |
| landuse=vineyard                                          |
| landuse=scrub                                             |
| landuse=grass                                             |
| natural=heath                                             |
| landuse=farmland                                          |
| landuse=farmyard                                          |
| landuse=landfill                                          |

| Polygon |                 |                                                  |
|---------|-----------------|--------------------------------------------------|
| 12xx    | adminareas      | Administrative boundaries                        |
| 15xx    | buildings       | Building outlines                                |
| 72xx    | landuse         | Forests, residential areas, industrial areas,... |
| 73xx    | protected_areas | Nature reserves, national parks                  |
| 82xx    | water           | Lakes, ...                                       |

esos “huecos” en OSM

## Desarrollo del Código:

[1]:

```
import osmnx as ox
import geopandas as gpd
print("Librerías cargadas correctamente")
```

Librerías cargadas correctamente

[2]:

```
import osmnx as ox

# Descargar las carreteras de la Comunidad de Madrid
print("Descargando datos de Madrid... esto puede tardar unos minutos")
vias = ox.graph_from_place("Comunidad de Madrid, Spain", network_type="")
print("Datos descargados correctamente")
```

Descargando datos de Madrid... esto puede tardar unos minutos  
Datos descargados correctamente

[3]:

```
# Convertir los datos a un formato que podamos trabajar
vias_gdf = ox.graph_to_gdfs(vias, nodes=False, edges=True)

# Ver cuántos tramos tenemos y qué columnas hay
print(f"Total de tramos: {len(vias_gdf)}")
print(f"Columnas disponibles: {list(vias_gdf.columns)}")
```

Total de tramos: 271072  
Columnas disponibles: ['osmid', 'highway', 'lanes', 'maxspeed', 'name', 'oneway', 'ref', 'reversed', 'length', 'geometry', 'bridge', 'junction', 'tunnel', 'access', 'width', 'est\_width']

[ ]:

Tiene 271.072 tramos y ya podemos ver que existe la columna maxspeed y la columna highway y ahora vere cuántos tramos tienen la velocidad y cuántos no

```
[4]: # Ver cuántos tramos tienen maxspeed y cuántos no
con_velocidad = vias_gdf[vias_gdf["maxspeed"].notna()]
sin_velocidad = vias_gdf[vias_gdf["maxspeed"].isna()]

print(f"Tramos CON velocidad: {len(con_velocidad)}")
print(f"Tramos SIN velocidad: {len(sin_velocidad)}")
print(f"Porcentaje sin velocidad: {round(len(sin_velocidad)/len(vias_gdf)*100, 1)}%")
```

Tramos CON velocidad: 68198  
Tramos SIN velocidad: 202874  
Porcentaje sin velocidad: 74.8%

Esto es exactamente el problema del proyecto.

El 74.8% de los tramos no tienen velocidad.

Ahora aplicare la inferencia por tipo de vía

```
[5]: # Tabla de velocidades por tipo de vía
velocidades = {
    "motorway": 120, "motorway_link": 80,
    "trunk": 100, "trunk_link": 60,
    "primary": 90, "primary_link": 70,
    "secondary": 90, "secondary_link": 70,
    "tertiary": 50, "tertiary_link": 50,
    "unclassified": 50,
    "residential": 30,
    "living_street": 20,
    "service": 20,
    "track": 20
}

# Aplicar velocidad inferida donde no hay maxspeed
def inferir_velocidad(row):
    if row["maxspeed"] is not None and row["maxspeed"] == row["maxspeed"]:
        return row["maxspeed"] # ya tiene velocidad, respetar
    highway = row["highway"]
    if isinstance(highway, list):
        highway = highway[0] # a veces viene como lista
    return velocidades.get(highway, 50) # 50 por defecto si no se conoce

vias_gdf["maxspeed_inferida"] = vias_gdf.apply(inferir_velocidad, axis=1)
print("Velocidades inferidas correctamente")
print(vias_gdf[["highway", "maxspeed", "maxspeed_inferida"]].head(10))
```

Velocidades inferidas correctamente

| u        | v          | key | highway                   | maxspeed \ |
|----------|------------|-----|---------------------------|------------|
| 21741584 | 21741587   | 0   | motorway                  | 100        |
| 21741587 | 759985345  | 0   | motorway_link             | [40, 70]   |
|          | 310025091  | 0   | motorway                  | 100        |
| 21741595 | 1317348460 | 0   | [motorway_link, tertiary] | 40         |
|          | 310031145  | 0   | motorway                  | [120, 100] |
| 26387798 | 310030930  | 0   | motorway                  | 120        |
|          | 2138901502 | 0   | motorway_link             | 70         |
| 26388046 | 305294808  | 0   | motorway_link             | [40, 60]   |
|          | 305295292  | 0   | motorway                  | 100        |
| 26388231 | 313011964  | 0   | motorway                  | 100        |

| u        | v          | key | maxspeed_inferida |
|----------|------------|-----|-------------------|
| 21741584 | 21741587   | 0   | 100               |
| 21741587 | 759985345  | 0   | [40, 70]          |
|          | 310025091  | 0   | 100               |
| 21741595 | 1317348460 | 0   | 40                |
|          | 310031145  | 0   | [120, 100]        |
| 26387798 | 310030930  | 0   | 120               |
|          | 2138901502 | 0   | 70                |
| 26388046 | 305294808  | 0   | [40, 60]          |
|          | 305295292  | 0   | 100               |
| 26388231 | 313011964  | 0   | 100               |

Ya tiene la columna maxspeed\_inferida con velocidades asignadas a cada tramo. Ahora guardo todo como GeoJSON que es el formato en el que me han pedido trabajar

```
[6]: # Guardar como GeoJSON
vias_gdf["maxspeed_inferida"] = vias_gdf["maxspeed_inferida"].astype(str)
vias_gdf.to_file("velocidades_madrid.geojson", driver="GeoJSON")
print("Archivo guardado correctamente como velocidades_madrid.geojson")
```

Archivo guardado correctamente como velocidades\_madrid.geojson



Por ahora he hecho esto:

- ☐ Instalado todo el entorno de Python
- ☐ Descargado los datos de Madrid
- ☐ Cargado 271.072 tramos de carretera
- ☐ Descubierta que el 74.8% no tiene velocidad que era el problema del proyecto
- ☐ He aplicado la inferencia por tipo de vía
- ☐ Guardado el resultado como velocidades\_madrid.geojson

```
# Descargar polígonos de landuse de Madrid
print("Descargando polígonos de landuse... puede tardar unos minutos")
landuse = ox.features_from_place(
    "Comunidad de Madrid, Spain",
    tags={"landuse": True}
)
print(f"Polígonos descargados: {len(landuse)}")
print(f"Tipos de landuse encontrados: {landuse['landuse'].unique()}")
```

Descargando polígonos de landuse... puede tardar unos minutos

Polígonos descargados: 34707

Tipos de landuse encontrados: <StringArray>

```
[
    'military',
    'quarry',
    'residential',
    'recreation_ground',
    'retail',
    'village_green',
    'greenfield',
    'religious',
    'winter_sports',
    'farmland',
    'orchard',
    'basin',
    'allotments',
    'government',
    'landfill',
    'depot',
    'prison',
    'plant_nursery',
    'services',
    'recycling',
    'salt_pond',
    'fairground',
    'courthouse',
    'wasteland',
    'sand',
    'water_storage',
    'animal_keeping',
    'concrete',
    'industrial',
    'forest',
    'apiary',
    'brownfield',
    'education',
    'cemetery',
    'meadow',
    'grass',
    'farmyard',
    'commercial',
    'railway',
    'flowerbed',
    'construction',
    'governmental',
    'healthcare',
    'common',
    'reservoir',
    'parking',
    'greenhouse_horticulture',
    'institutional',
    'vineyard',
    'garages',
    'traffic_island',
    'shanty_town',
    'aquaculture',
    'administration',
    'greenery',
    'emergency']
```

Length: 56, dtype: str

Tengo que descargar los polígonos de landuse de Madrid

La versión nueva de osmnx cambió el nombre de esa función entonces tuve que buscar como se llamaba ahora

34.707 polígonos descargados con 56 tipos distintos de landuse.

Ahora tengo que definir cuáles son urbanos y aplicar la máscara.

```
[9]: # Definir zonas urbanas y rurales
zonas_urbanas = ["residential", "commercial", "industrial", "retail",
                 "education", "healthcare", "construction", "garages"]

# Filtrar solo los polígonos urbanos y quedarnos solo con polígonos (no puntos)
landuse_urbano = landuse[
    (landuse["landuse"].isin(zonas_urbanas)) &
    (landuse.geometry.geom_type.isin(["Polygon", "MultiPolygon"]))
].copy()

print(f"Polígonos urbanos: {len(landuse_urbano)}")

# Asegurarse de que ambas capas tienen el mismo sistema de coordenadas
landuse_urbano = landuse_urbano.to_crs(vias_gdf.crs)

# Hacer el spatial join
vias_con_contexto = gpd.sjoin(
    vias_gdf,
    landuse_urbano[["landuse", "geometry"]],
    how="left",
    predicate="intersects"
)

# Marcar si cada tramo está en zona urbana o no
vias_con_contexto["es_urbano"] = vias_con_contexto["landuse"].notna()

print(f"Tramos en zona urbana: {vias_con_contexto['es_urbano'].sum()}")
print(f"Tramos en zona rural: {(~vias_con_contexto['es_urbano']).sum()}")

Polígonos urbanos: 13133
Tramos en zona urbana: 239400
Tramos en zona rural: 54020
```

La máscara urbana está funcionando bien

Se ve que hay 239.400 tramos en zona urbana y 54.020 en zona rural.

Ahora aplicare las velocidades según el contexto

```
[10]: # Velocidades según contexto urbano o interurbano
def velocidad_urbana(highway):
    if isinstance(highway, list):
        highway = highway[0]
    velocidades = {
        "motorway": 120, "motorway_link": 80,
        "trunk": 80, "trunk_link": 50,
        "primary": 50, "primary_link": 50,
        "secondary": 50, "secondary_link": 50,
        "tertiary": 30, "tertiary_link": 30,
        "unclassified": 30,
        "residential": 30,
        "living_street": 20,
        "service": 20,
        "track": 20
    }
    return velocidades.get(highway, 30)

def velocidad_interurbana(highway):
    if isinstance(highway, list):
        highway = highway[0]
    velocidades = {
        "motorway": 120, "motorway_link": 80,
        "trunk": 100, "trunk_link": 60,
        "primary": 90, "primary_link": 70,
        "secondary": 90, "secondary_link": 70,
        "tertiary": 70, "tertiary_link": 50,
        "unclassified": 50,
        "residential": 30,
        "living_street": 20,
        "service": 20,
        "track": 20
    }
    return velocidades.get(highway, 50)

# Aplicar velocidad según contexto
def inferir_con_contexto(row):
    # Si ya tiene maxspeed, respetarlo
    if row["maxspeed"] is not None and row["maxspeed"] == row["maxspeed"]:
        return row["maxspeed"]
    # Si no tiene, inferir según contexto
    if row["es_urbano"]:
        return velocidad_urbana(row["highway"])
    else:
        return velocidad_interurbana(row["highway"])

vias_con_contexto["maxspeed_final"] = vias_con_contexto.apply(inferir_con_contexto, axis=1)
print("Velocidades aplicadas correctamente")
print(vias_con_contexto[["highway", "es_urbano", "maxspeed", "maxspeed_final"]].head(10))
```

Velocidades aplicadas correctamente

| u        | v          | key | highway                   | es_urbano | maxspeed \ |
|----------|------------|-----|---------------------------|-----------|------------|
| 21741584 | 21741587   | 0   | motorway                  | False     | 100        |
| 21741587 | 759985345  | 0   | motorway_link             | False     | [40, 70]   |
|          | 310025091  | 0   | motorway                  | False     | 100        |
| 21741595 | 1317348460 | 0   | [motorway_link, tertiary] | True      | 40         |
|          | 310031145  | 0   | motorway                  | False     | [120, 100] |
| 26387798 | 310030930  | 0   | motorway                  | False     | 120        |
|          | 2138901502 | 0   | motorway_link             | False     | 70         |
| 26388046 | 305294808  | 0   | motorway_link             | False     | [40, 60]   |
|          | 305295292  | 0   | motorway                  | False     | 100        |
| 26388231 | 313011964  | 0   | motorway                  | False     | 100        |

| u        | v          | key | maxspeed_final |
|----------|------------|-----|----------------|
| 21741584 | 21741587   | 0   | 100            |
| 21741587 | 759985345  | 0   | [40, 70]       |
|          | 310025091  | 0   | 100            |
| 21741595 | 1317348460 | 0   | 40             |
|          | 310031145  | 0   | [120, 100]     |
| 26387798 | 310030930  | 0   | 120            |
|          | 2138901502 | 0   | 70             |
| 26388046 | 305294808  | 0   | [40, 60]       |
|          | 305295292  | 0   | 100            |
| 26388231 | 313011964  | 0   | 100            |

Se ve que esta funcionando por que se ve la columna `es_urbano` con `True` o `False` y la columna `maxspeed_final` con la velocidad asignada según el contexto. Ahora guardo el resultado final como GeoJSON.

```
[11]: # Guardar resultado final
vias_con_contexto["maxspeed_final"] = vias_con_contexto["maxspeed_final"].astype(str)

# Quedarnos solo con las columnas importantes
resultado = vias_con_contexto[["highway", "maxspeed", "es_urbano", "maxspeed_final", "geometry"]].copy()

# Eliminar duplicados que puede haber generado el spatial join
resultado = resultado[~resultado.index.duplicated(keep="first")]

resultado.to_file("velocidades_madrid_final.geojson", driver="GeoJSON")

print(f"Archivo guardado: velocidades_madrid_final.geojson")
print(f"Total tramos: {len(resultado)}")
print(f"Tramos con velocidad original OSM: {resultado['maxspeed'].notna().sum()}")
print(f"Tramos con velocidad inferida: {resultado['maxspeed'].isna().sum()}")

Archivo guardado: velocidades_madrid_final.geojson
Total tramos: 271072
Tramos con velocidad original OSM: 68198
Tramos con velocidad inferida: 202874
```

El dataset final está completo:

- 271.072 tramos en total con velocidad para todos
  - 68.198 con velocidad real de OSM
  - 202.874 con velocidad inferida mediante la máscara urbana
1. Descargue los datos de OSM con 271.072 tramos
  2. Identifique que el 74.8% no tenía velocidad
  3. Descargue 34.707 polígonos de landuse
  4. Aplique la máscara urbana para que cada tramo supiese si está en ciudad o en campo
  5. Asigne velocidades diferenciadas según el contexto urbano/rural
  6. Guarde el resultado final en GeoJSON

Antes (solo datos de OSM): Aprox 25% de las carreteras

- 68.198 tramos con velocidad
- 202.874 tramos sin velocidad (aparecían en blanco)

Después (con solución de poligonos): 100% de las carreteras

- 271.072 tramos con velocidad
- 0 tramos sin velocidad

No son velocidades al azar, cada tramo recibe una velocidad lógica según dos cosas: su tipo de vía (motorway, tertiary, residential, etc.) y si está en ciudad o en el campo. Por ejemplo en Mirasierra que es una tertiary en ciudad ya no recibe 70 km/h sino 30 km/h, que es lo correcto.

```

import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 8))

# MAPA 1 - Antes (solo datos OSM)
ax1.set_facecolor("#f0f0f0")
vias_gdf[vias_gdf["maxspeed"].isna()].plot(ax=ax1, color="#dddddd", linewidth=0.3)
vias_gdf[vias_gdf["maxspeed"].notna()].plot(ax=ax1, color="#e74c3c", linewidth=0.5)
ax1.set_title("ANTES\nSolo datos OSM", fontsize=14, fontweight="bold")
ax1.axis("off")
parche1 = mpatches.Patch(color="#e74c3c", label=f"Con velocidad: {len(con_velocidad)}")
parche2 = mpatches.Patch(color="#dddddd", label=f"Sin velocidad: {len(sin_velocidad)}")
ax1.legend(handles=[parche1, parche2], loc="lower left", fontsize=9)

# MAPA 2 - Después (con máscara urbana)
ax2.set_facecolor("#f0f0f0")
resultado[resultado["es_urbano"] == True].plot(ax=ax2, color="#e74c3c", linewidth=0.3)
resultado[resultado["es_urbano"] == False].plot(ax=ax2, color="#f39c12", linewidth=0.5)
ax2.set_title("DESPUÉS\nCon máscara urbana", fontsize=14, fontweight="bold")
ax2.axis("off")
parche3 = mpatches.Patch(color="#e74c3c", label=f"Zona urbana: {(resultado['es_urbano']==True).sum()}")
parche4 = mpatches.Patch(color="#f39c12", label=f"Zona rural: {(resultado['es_urbano']==False).sum()}")
ax2.legend(handles=[parche3, parche4], loc="lower left", fontsize=9)

plt.suptitle("Comunidad de Madrid - Inferencia de Velocidades Máximas",
             fontsize=16, fontweight="bold", y=1.02)
plt.tight_layout()
plt.savefig("comparacion_antes_despues.png", dpi=150, bbox_inches="tight")
plt.show()
print("Mapa guardado como comparacion_antes_despues.png")

```

## Mapa comparando el antes y el después

